

Lambda Expressions in C# with Examples

In this article, I am going to discuss the **Lambda Expressions in C#** with Examples. Please read our previous article where we discussed the **Anonymous Methods in C#** with Examples. As part of this article, we are going to discuss the following pointers in detail.

1. **What are Lambda Expressions in C#?**
2. **Why do we need Lambda Expressions?**
3. **How to create Lambda Expression in C#?**
4. **Examples of using Lambda Expression.**

What are Lambda Expressions in C#?

The Lambda Expression in C# is the shorthand for writing the Anonymous Function. So, we can say that the Lambda Expression is nothing but to simplify the anonymous function in C# and we also discussed that Anonymous Functions are related to delegate and they are created by using the delegate keyword.

So, what we will do is, first we will create one example using delegate and using a named block i.e. function, and then we will convert the same example using the anonymous method and finally we will discuss what are the problems with the anonymous method and how we can overcome the same using lambda expression.

Example to Understand Delegate using Method in C#:

In the below example, we are creating one delegate and one method with the same signature and then registering the method with the delegate instance and when we invoke the delegate, the method which is registered with the delegate is going to be executed.

```
using System;
namespace LambdaExpressionDemo
{
    public class LambdaExpression
    {
        public delegate string GreetingsDelegate(string name);
        static void Main(string[] args)
        {
            GreetingsDelegate obj = new GreetingsDelegate(LambdaExpression.Gre
            string GreetingsMessage = obj.Invoke("Pranaya");
            Console.WriteLine(GreetingsMessage);
            Console.ReadKey();
        }
        public static string Greetings(string name)
        {
            return "Hello @" + name + " welcome to Dotnet Tutorials";
        }
    }
}
```

Output: **Hello @Pranaya welcome to Dotnet Tutorials**

Delegate using Anonymous Method in C#:

In the previous example, we used a named block while creating the delegate instance. Instead of a named block, we can also give an unnamed block which is called Anonymous Method. The anonymous methods are created using the delegate keyword and when we invoke the delegate, the anonymous method is going to be executed. Please have a look at the following example. This is the same example as the previous example and this example is also going to give you the same output. Here, instead of a named block, we are using an unnamed block.

```
using System;
namespace LambdaExpressionDemo
{
    public class LambdaExpression
    {
        public delegate string GreetingsDelegate(string name);
        static void Main(string[] args)
        {
            GreetingsDelegate obj = delegate (string name)
            {
                return "Hello @" + name + " welcome to Dotnet Tutorials";
            };
            string GreetingsMessage = obj.Invoke("Pranaya");
            Console.WriteLine(GreetingsMessage);
            Console.ReadKey();
        }
    }
}
```

Output: Hello @Pranaya welcome to Dotnet Tutorials

Here, you need to understand two things. As we are using the Anonymous Method in C# for lesser writing, then why do we need to use the delegate keyword? And the second thing is as the delegate knows about the return type and input parameter type it accepts, then why do we need to explicitly specify the parameter type the delegate accepts?

Why Delegate Keyword and String Data Type

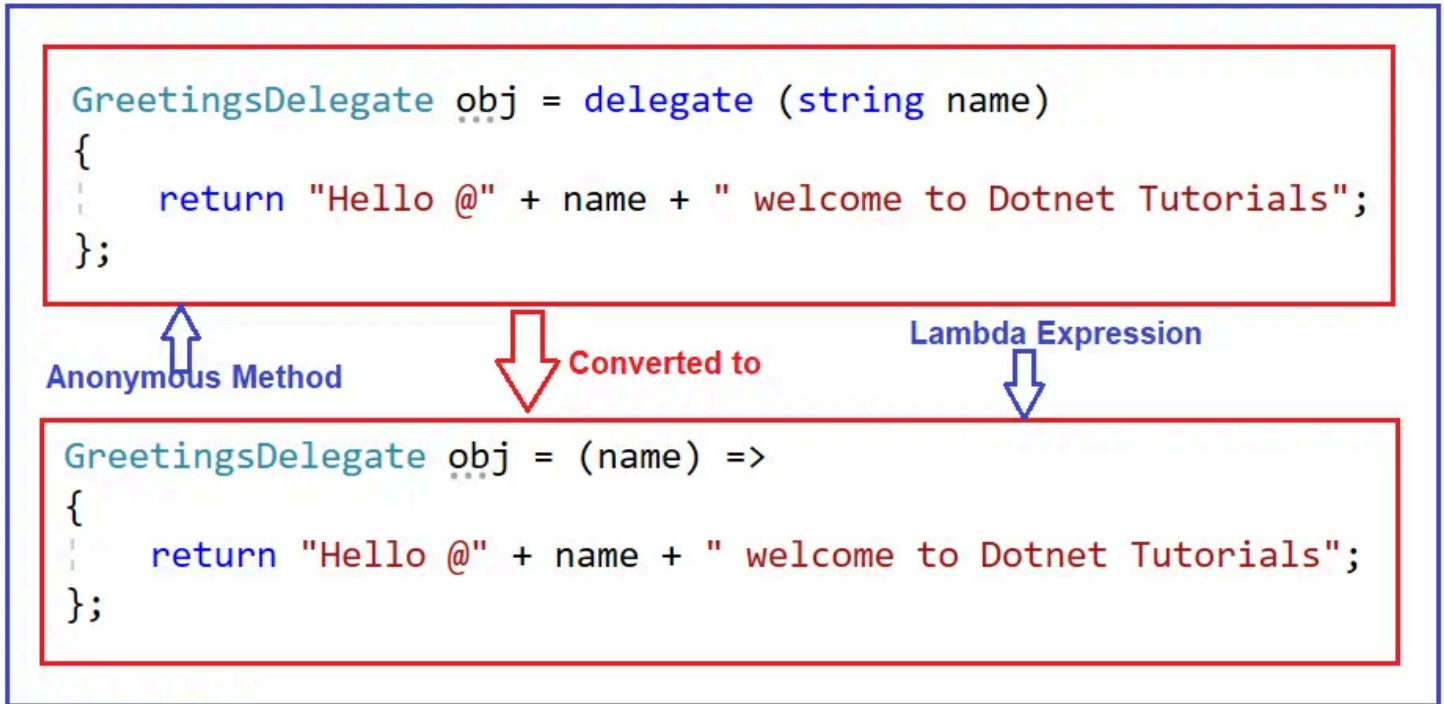


```
GreetingsDelegate obj = delegate (string name)
{
    return "Hello @" + name + " welcome to Dotnet Tutorials";
};
```

In our example, how we can eliminate the delegate keyword and string data type of the input name parameter? We can overcome this by using Lambda Expressions which was introduced in C#3.0.

How to Create Lambda Expressions in C#?

To create a lambda expression in C#, we need to specify the input parameters (if any) on the left side of the lambda operator `=>`, and we need to put the expression or statement block within the open and close curly braces. For a better understanding, please have a look at the below image which shows how to convert an anonymous method to a lambda expression. Here, you can see, we have replaced the delegate keyword with lambda operator (`=>`) and we have also removed the data type of the parameter as the delegate knows the type of the parameter.



Example to Understand Lambda Expressions in C#:

Let us rewrite the previous example using the Lambda Expression in C#. And this time also, you will get the same output.

```
using System;
namespace LambdaExpressionDemo
{
    public class LambdaExpression
    {
        public delegate string GreetingsDelegate(string name);
        static void Main(string[] args)
        {
            GreetingsDelegate obj = (name) =>
            {
                return "Hello @" + name + " welcome to Dotnet Tutorials";
            };
            string GreetingsMessage = obj.Invoke("Pranaya");
            Console.WriteLine(GreetingsMessage);
            Console.ReadKey();
        }
    }
}
```

```
    public static string Greetings(string name)
    {
        return "Hello @" + name + " welcome to Dotnet Tutorials";
    }
}
```

Output: **Hello @Pranaya welcome to Dotnet Tutorials**

Note: In our [LINQ Tutorials](#), we will explore Lambda Expressions in detail with Real-time Examples. This is because Lambda Expressions are extensively used with LINQ queries.

In the next article, I am going to discuss [Events in C#](#) with Examples. Here, in this article, I try to explain **Lambda Expressions in C# with Examples**. I hope now you understood how to create and work with Lambda Expression in C#.